

Geminio OS - API Documentation

Overview

Geminio OS provides a comprehensive tRPC API for all platform features. The API is type-safe, fully documented, and production-ready.

Base URL

Plain Text

Development: `http://localhost:3000`

Production: `https://api.geminio-os.app`

Authentication

All API requests require authentication via OAuth or JWT token.

TypeScript

```
// OAuth Flow
const token = await getOAuthToken( );
const client = createTRPCClient({
  links: [
    httpBatchLink({
      url: 'http://localhost:3000/trpc',
      headers: {
        Authorization: `Bearer ${token}`,
      },
    } ),
  ],
});
```

API Endpoints

1. Builder Routers

App Builder

TypeScript

```
// Create app project
const app = await trpc.builders.createAppProject.mutate({
  name: 'My App',
  screens: ['Home', 'Settings'],
  components: ['Button', 'Card'],
});

// Generate React Native code
const code = await trpc.builders.generateAppCode.mutate({
  projectId: app.id,
  framework: 'react-native',
});

// Save project
await trpc.builders.saveProject.mutate({
  projectId: app.id,
  data: app,
});
```

Website Builder

TypeScript

```
// Create website project
const website = await trpc.builders.createWebsiteProject.mutate({
  name: 'My Website',
  template: 'landing-page',
  domain: 'mysite.com',
});

// Generate HTML/CSS
const html = await trpc.builders.generateWebsiteCode.mutate({
  projectId: website.id,
});

// Deploy website
await trpc.builders.deployWebsite.mutate({
  projectId: website.id,
});
```

Workflow Builder

TypeScript

```
// Create workflow
const workflow = await trpc.builders.createWorkflow.mutate({
  name: 'Email Notification',
  trigger: 'on_event',
  actions: [
    { type: 'send_email', recipient: 'user@example.com' },
  ],
});

// Execute workflow
await trpc.builders.executeWorkflow.mutate({
  workflowId: workflow.id,
  data: { event: 'user_signup' },
});
```

Database Builder

TypeScript

```
// Create database schema
const schema = await trpc.builders.createDatabaseSchema.mutate({
  name: 'users_db',
  tables: [
    {
      name: 'users',
      columns: [
        { name: 'id', type: 'uuid', primary: true },
        { name: 'email', type: 'string', unique: true },
      ],
    },
  ],
});

// Generate SQL
const sql = await trpc.builders.generateDatabaseSQL.mutate({
  schemaId: schema.id,
  dialect: 'mysql',
});
```

AI Agent Builder

TypeScript

```
// Create AI agent
const agent = await trpc.builders.createAIAgent.mutate({
```

```
    name: 'Customer Support Bot',
    model: 'gpt-4',
    instructions: 'You are a helpful customer support agent',
    tools: ['search_knowledge_base', 'create_ticket'],
  });

// Chat with agent
const response = await trpc.builders.chatWithAgent.mutate({
  agentId: agent.id,
  message: 'How do I reset my password?',
});
```

dApp Builder

TypeScript

```
// Create blockchain app
const dapp = await trpc.builders.createDApp.mutate({
  name: 'My NFT Marketplace',
  blockchain: 'ethereum',
  contracts: ['ERC721', 'ERC20'],
});

// Deploy smart contract
await trpc.builders.deploySmartContract.mutate({
  dappId: dapp.id,
  contract: contractCode,
});
```

2. User Management

TypeScript

```
// Get current user
const user = await trpc.auth.getMe.query();

// Update profile
await trpc.user.updateProfile.mutate({
  bio: 'Software Developer',
  avatar: 'https://...',
  theme: 'dark',
} );

// Get user projects
const projects = await trpc.user.getProjects.query();
```

```
// Get user tasks
const tasks = await trpc.user.getTasks.query({
  status: 'in_progress',
});
```

3. Project Management

TypeScript

```
// Create project
const project = await trpc.projects.create.mutate({
  name: 'My Project',
  type: 'app',
  description: 'A new mobile app',
});

// Get project details
const details = await trpc.projects.getById.query({ id: projectId });

// Update project
await trpc.projects.update.mutate({
  id: projectId,
  name: 'Updated Name',
  status: 'active',
});

// Delete project
await trpc.projects.delete.mutate({ id: projectId });

// List all projects
const allProjects = await trpc.projects.list.query({
  limit: 10,
  offset: 0,
});
```

4. Task Management

TypeScript

```
// Create task
const task = await trpc.tasks.create.mutate({
  title: 'Implement login screen',
  description: 'Add OAuth login',
  priority: 'high',
  dueDate: new Date('2024-12-31'),
});
```

```
// Update task
await trpc.tasks.update.mutate({
  id: taskId,
  status: 'done',
});

// Get tasks
const tasks = await trpc.tasks.list.query({
  status: 'in_progress',
});
```

5. Goal Tracking

TypeScript

```
// Create goal
const goal = await trpc.goals.create.mutate({
  title: 'Complete Geminio OS',
  description: 'Build all 14 phases',
  target: 100,
});

// Update progress
await trpc.goals.updateProgress.mutate({
  id: goalId,
  progress: 75,
});

// Get goals
const goals = await trpc.goals.list.query();
```

6. Chat & Messages

TypeScript

```
// Send message
const message = await trpc.messages.send.mutate({
  conversationId: 'conv-1',
  content: 'Hello, how can I help?',
  role: 'user',
});

// Get conversation
const messages = await trpc.messages.getConversation.query({
  conversationId: 'conv-1',
});
```

```
});

// Chat with AI
const aiResponse = await trpc.messages.chatWithAI.mutate({
  content: 'How do I build an app?',
});
```

7. Media Management

TypeScript

```
// Upload media
const media = await trpc.media.upload.mutate({
  file: fileBlob,
  type: 'image',
  name: 'screenshot.png',
});

// Get media
const mediaList = await trpc.media.list.query();

// Delete media
await trpc.media.delete.mutate({ id: mediaId });
```

8. Wallet & Transactions

TypeScript

```
// Get wallet
const wallet = await trpc.wallet.getBalance.query();

// Create transaction
const transaction = await trpc.wallet.createTransaction.mutate({
  type: 'expense',
  category: 'Food',
  amount: 25.50,
  description: 'Lunch',
});

// Get transactions
const transactions = await trpc.wallet.getTransactions.query({
  startDate: new Date('2024-01-01'),
  endDate: new Date('2024-12-31'),
});
```

```
// Get budgets
const budgets = await trpc.wallet.getBudgets.query();
```

9. Notifications

TypeScript

```
// Get notifications
const notifications = await trpc.notifications.list.query();

// Mark as read
await trpc.notifications.markAsRead.mutate({ id: notificationId });

// Delete notification
await trpc.notifications.delete.mutate({ id: notificationId });
```

10. Marketplace

TypeScript

```
// Get marketplace items
const items = await trpc.marketplace.list.query({
  type: 'template',
  limit: 20,
});

// Get item details
const item = await trpc.marketplace.getById.query({ id: itemId });

// Purchase item
const purchase = await trpc.marketplace.purchase.mutate({
  itemId: itemId,
});

// Publish item
await trpc.marketplace.publish.mutate({
  name: 'My Template',
  type: 'template',
  price: 29.99,
  description: 'A great template',
});
```

11. Admin Operations

TypeScript

```
// Get all users (admin only)
const users = await trpc.admin.getUsers.query();

// Suspend user
await trpc.admin.suspendUser.mutate({ userId: userId });

// Get audit logs
const logs = await trpc.admin.getAuditLogs.query();

// Get system health
const health = await trpc.admin.getSystemHealth.query();
```

12. Security & Permissions

TypeScript

```
// Get permissions
const permissions = await trpc.security.getPermissions.query();

// Request permission
await trpc.security.requestPermission.mutate({
  type: 'camera',
});

// Revoke permission
await trpc.security.revokePermission.mutate({
  type: 'location',
});

// Get security alerts
const alerts = await trpc.security.getAlerts.query();
```

Error Handling

All API errors follow a consistent format:

TypeScript

```
{
  code: 'BAD_REQUEST' | 'UNAUTHORIZED' | 'FORBIDDEN' | 'NOT_FOUND' | 'INTERNAL_SERVER_ERROR',
  message: 'Human-readable error message',
  data?: {
    field?: string,
    details?: any,
  },
}
```

```
}  
}
```

Example error handling:

TypeScript

```
try {  
  await trpc.projects.create.mutate({ name: '' });  
} catch (error) {  
  if (error.code === 'BAD_REQUEST') {  
    console.error('Validation error:', error.message);  
  } else if (error.code === 'UNAUTHORIZED') {  
    // Redirect to login  
  }  
}
```

Rate Limiting

API requests are rate-limited:

- **Authenticated users:** 1000 requests/hour
- **Unauthenticated:** 100 requests/hour
- **Admin:** Unlimited

Rate limit headers:

Plain Text

```
X-RateLimit-Limit: 1000  
X-RateLimit-Remaining: 999  
X-RateLimit-Reset: 1234567890
```

Pagination

List endpoints support pagination:

TypeScript

```
// Get paginated results  
const results = await trpc.projects.list.query({  
  limit: 20,  
  offset: 0,  
  sort: 'createdAt',  
});
```

```

    order: 'desc',
  });

// Response
{
  items: [...],
  total: 100,
  limit: 20,
  offset: 0,
  hasMore: true,
}

```

Webhooks

Subscribe to events:

TypeScript

```

// Register webhook
await trpc.webhooks.register.mutate({
  url: 'https://myapp.com/webhook',
  events: ['project.created', 'task.completed'],
});

// Webhook payload
{
  event: 'project.created',
  timestamp: '2024-01-15T10:30:00Z',
  data: {
    projectId: 'proj-123',
    name: 'My Project',
  }
}

```

SDK Usage

TypeScript/JavaScript

TypeScript

```

import { createTRPCClient, httpBatchLink } from '@trpc/client';
import type { AppRouter } from '@server/routers';

const trpc = createTRPCClient<AppRouter>({

```

```
links: [
  httpBatchLink({
    url: 'http://localhost:3000/trpc',
  }),
],
});

// Use API
const projects = await trpc.projects.list.query();
```

React

TypeScript

```
import { trpc } from '@lib/trpc';

function ProjectList() {
  const { data: projects } = trpc.projects.list.useQuery();

  return (
    <div>
      {projects?.map(p => (
        <div key={p.id}>{p.name}</div>
      ))}
    </div>
  );
}
```

API Versioning

Current version: v1

Future versions will be available at:

- /trpc/v2
- /trpc/v3

Support

For API support:

- Documentation: <https://docs.geminio-os.app>
- Issues: <https://github.com/geminio-os/issues>
- Email: api-support@geminio-os.app